STOCK PRICE PREDICTION USING APACHE AIRFLOW

(Pair 15)

Varun Reddy Bhumi Reddy
San Jose State University
San Jose, CA, USA
varunreddy.bhumireddy@sjsu.edu

Ankit Sharma
San Jose State University
San Jose, CA, USA
ankit.sharma@sjsu.edu

***Proposal*— The Stock Price Prediction System is designed to automate data ingestion, processing, and machine learning-based forecasting of stock prices. The system integrates an Extract, Transform, and Load (ETL) pipeline using Apache Airflow, a relational database for structured data storage, and a predictive modeling pipeline utilizing machine learning techniques. The purpose of this system is to provide real-time insights into stock market trends, allowing traders and investors to make informed financial decisions.**

Keywords—Apache Airflow, Snowflake, SQL,Docker,DAG's

I. PROBLEM STATEMENT

The proposed application system is a Stock Price Prediction System that automates data ingestion, processing, and machine learning forecasting for stock prices. The system is crucial for traders, investors, and financial analysts who require real-time stock market predictions.

A database and data pipelines are essential components of this system because:

- A database ensures structured storage, retrieval, and management of historical and real-time stock data.
- Data pipelines automate the ETL (Extract, Transform, Load) process, ensuring continuous data updates and enabling machine learning predictions on fresh data.
- The integration of machine learning forecasting provides predictive insights based on historical trends, improving investment decisions.

II. SOLUTION REQUIREMENTS

System Capabilities:

- Stock Data Ingestion: Automatically fetch stock prices for companies like Google (GOOGL) and Apple (AAPL) using the yfinance API.
- ETL Data Pipeline: Process and store fetched stock data into a relational database using Apache Airflow.
- Docker Desktop is used to containerize and deploy.
- Snowflake is utilized as a cloud-based data warehouse for scalable storage and fast retrieval of stock market data, supporting analytics and machine learning workflows.
- Machine Learning Forecasting Pipeline: Predict future stock prices using historical trends and store results in the database.

System Limitations:

- The model's accuracy is constrained by the quality of available historical data.
- Real-time data acquisition depends on the reliability of the yfinance API.

System Usage:

- Traders and analysts access historical and predicted stock prices via SQL queries.
- The system runs scheduled Airflow DAGs for continuous data updates and forecasting.
- Data visualization tools integrate with the database to present results.
- Docker Desktop is used to containerize and deploy the application efficiently, ensuring consistency across different environments.
- Data warehousing tools integrate with the database to present results. Predictions are subject to market fluctuations and external economic conditions. Maintaining the Integrity of the Specifications

III. FUNCTIONAL ANALYSIS

The system consists of the following key functional components:

3.1 Data Ingestion (ETL Pipeline)

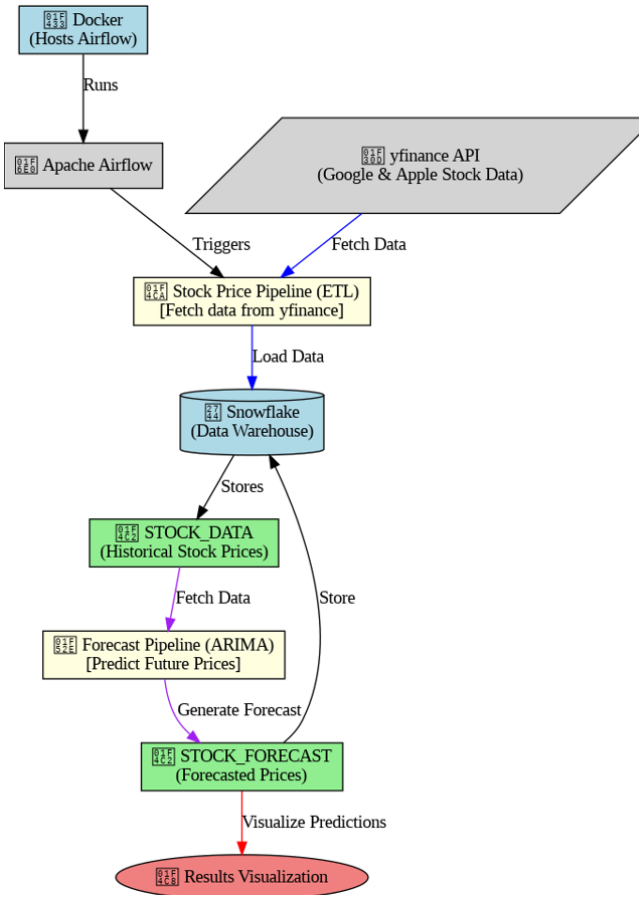
- Fetch real-time stock data from yfinance API.
- Transform and clean the data, ensuring proper formatting for database storage in SnowFlake.
- Load data into a structured SQL table on SnowFlake.
- Run as an Airflow DAG, executing at scheduled intervals.

3.2 Machine Learning Forecasting Pipeline

- Fetch historical stock data from the database.
- Train a predictive model (ARIMA) using the historical data on SnowFlake.
- Generate stock price forecasts for future time frames.
- Store predictions in a forecasting table.

- Union the original ETL table with the forecasted results to create a final combined table.
- Run as an Airflow DAG, executing at scheduled intervals.

3.3 System Diagram:



3.4 Database & Data Pipeline Interactions

COMPONENT	FUNCTION
SnowFlake	Provides scalable cloud-based storage, enabling fast retrieval and analytics for stock market data
Airflow	Automates data ingestion and forecasting processes, scheduling ETL and ML tasks efficiently
Docker Desktop	Ensures containerized deployment, maintaining consistency across different environments
Airflow DAGs	Define workflows for ETL and machine learning pipelines, ensuring scheduled and dependency-based execution

IV. Tables Structure, Screenshots, Python Codes, and SQL Queries

Git Link: CLICK

4.1 STOCK_DATA_PIPELINE CODE:

```
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from airflow.models import Variable
from datetime import datetime
import yfinance as yf
import pandas as pd
import snowflake.connector
import os
import logging


# Fetch Snowflake credentials from Airflow Variables
SNOWFLAKE_USER = Variable.get("snowflake_user")
SNOWFLAKE_PASSWORD = Variable.get("snowflake_password")
SNOWFLAKE_ACCOUNT = Variable.get("snowflake_account")
SNOWFLAKE_WAREHOUSE = Variable.get("snowflake_warehouse")
SNOWFLAKE_DATABASE = Variable.get("snowflake_database")
SNOWFLAKE_SCHEMA = Variable.get("snowflake_schema")


# Local file path for temporary CSV storage
LOCAL_FILE_PATH = "stock_data.csv"


# Define default_args for Airflow
default_args = {
    'owner': 'airflow',
    'start_date': datetime(),
    'retries': 1,
}


# Define the Airflow DAG
dag = DAG(
    'stock_price_pipeline',
    default_args=default_args,
    schedule_interval='@daily',
```

```
    catchup=False,
)

# Function to create Snowflake table
def create_snowflake_table():
    try:
        conn = snowflake.connector.connect(
            user=SNOWFLAKE_USER,
            password=SNOWFLAKE_PASSWORD,
            account=SNOWFLAKE_ACCOUNT,
            warehouse=SNOWFLAKE_WAREHOUSE,
            database=SNOWFLAKE_DATABASE,
            schema=SNOWFLAKE_SCHEMA
        )
        cursor = conn.cursor()

        cursor.execute(f"""
            CREATE OR REPLACE TABLE {SNOWFLAKE_DATABASE}.{SNOWFLAKE_SCHEMA}.STOCK_DATA (
                SYMBOL STRING,
                DATE DATE,
                OPEN FLOAT,
                CLOSE FLOAT,
                LOW FLOAT,
                HIGH FLOAT,
                VOLUME INTEGER
            )
        """)

        logging.info("✅ Table STOCK_DATA created successfully in Snowflake!")
        cursor.close()
        conn.close()
    except Exception as e:
        logging.error(f"❌ Error in create_snowflake_table: {str(e)}")

# Function to fetch stock data using yfinance.download()
def fetch_stock_data():
    stocks = ["GOOGL", "AAPL"]

    try:
        # Download data for multiple stocks using yfinance
        df_list = []
        for stock in stocks:
            df = yf.download(stock, period="180d", interval="1d")
            df.reset_index(inplace=True)
            df["Symbol"] = stock  # Add stock symbol column
            df_list.append(df)

        # Combine data for all stocks into a single DataFrame
        df_final = pd.concat(df_list, ignore_index=True)

        # Rename columns to match Snowflake table schema
        df_final.rename(columns={
            "Date": "DATE",
            "Open": "OPEN",
            "Close": "CLOSE",
            "Low": "LOW",
            "High": "HIGH",
            "Volume": "VOLUME"
        }, inplace=True)

        # Ensure DATE column is in correct format
        df_final["DATE"] = pd.to_datetime(df_final["DATE"]).dt.strftime('%Y-%m-%d')

        # Save to CSV for staging
        df_final.to_csv(LOCAL_FILE_PATH, index=False, header=True)

        logging.info(f"✅ Stock data saved to {LOCAL_FILE_PATH}")
    except Exception as e:
        logging.error(f"❌ Error in fetch_stock_data: {str(e)}")

# Function to load data into Snowflake using COPY INTO
def load_into_snowflake():
    try:
        conn = snowflake.connector.connect(
            user=SNOWFLAKE_USER,
```

```
            password=SNOWFLAKE_PASSWORD,
            account=SNOWFLAKE_ACCOUNT,
            warehouse=SNOWFLAKE_WAREHOUSE,
            database=SNOWFLAKE_DATABASE,
            schema=SNOWFLAKE_SCHEMA
        )
        cursor = conn.cursor()

        # Create a temporary stage in Snowflake
        stage_name = "TEMP_STAGE"
        cursor.execute(f"CREATE TEMPORARY STAGE {stage_name}")

        # Upload the CSV file to the temporary stage
        cursor.execute(f"PUT file://{LOCAL_FILE_PATH} @{stage_name}")

        # Use COPY INTO to load data into Snowflake efficiently
        cursor.execute(f"""
            COPY INTO {SNOWFLAKE_DATABASE}.{SNOWFLAKE_SCHEMA}.STOCK_DATA
            FROM @{stage_name}/{os.path.basename(LOCAL_FILE_PATH)}
            FILE_FORMAT = (
                TYPE = 'CSV',
                SKIP_HEADER = 1,
                FIELD_OPTIONALLY_ENCLOSED_BY = '"'
            )
        """)

        logging.info("✅ Data successfully loaded into Snowflake using COPY INTO!")
        cursor.close()
        conn.close()
    except Exception as e:
        logging.error(f"❌ Error in load_into_snowflake: {str(e)}")

# Function to run the full ETL pipeline
def run_pipeline():
    try:
        create_snowflake_table()
        fetch_stock_data()
        load_into_snowflake()
        logging.info("✅ ETL pipeline completed successfully!")
    except Exception as e:
        logging.error(f"❌ Error in run_pipeline: {str(e)}")

# Create Airflow tasks
run_stock_pipeline = PythonOperator(
    task_id='run_stock_pipeline',
    python_callable=run_pipeline,
    dag=dag,
)

run_stock_pipeline  # Start the pipeline
```

4.2 FORECAST DATA PIPELINE DAG CODE:

```
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from airflow.operators.dagrun_operator import TriggerDagRunOperator
from airflow.models import Variable
from datetime import datetime
import logging
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
import snowflake.connector
from airflow.decorators import task
from airflow.operators.dummy_operator import DummyOperator  # Correct import for DummyOperator

# Fetch Snowflake credentials from Airflow Variables
SNOWFLAKE_USER = Variable.get("snowflake_user")
SNOWFLAKE_PASSWORD = Variable.get("snowflake_password")
SNOWFLAKE_ACCOUNT = Variable.get("snowflake_account")
SNOWFLAKE_WAREHOUSE = Variable.get("snowflake_warehouse")
SNOWFLAKE_DATABASE = Variable.get("snowflake_database")
SNOWFLAKE_SCHEMA = Variable.get("snowflake_schema")

# Define default_args for Airflow
```

```
default_args = {
    'owner': 'airflow',
    'start_date': datetime.today(),  # DAG starts from today
    'retries': 1,
}

# Define the Airflow DAG
dag = DAG(
    'forecast_pipeline',
    default_args=default_args,
    schedule_interval='@daily',  # Runs daily
    catchup=False,
)

def return_snowflake_conn():
    """
    Return a Snowflake connection cursor using Airflow Variables.
    """
    conn = snowflake.connector.connect(
        user=SNOWFLAKE_USER,
        password=SNOWFLAKE_PASSWORD,
        account=SNOWFLAKE_ACCOUNT,
        warehouse=SNOWFLAKE_WAREHOUSE,
        database=SNOWFLAKE_DATABASE,
        schema=SNOWFLAKE_SCHEMA
    )
    return conn

@task(dag=dag)
def fetch_stock_data_from_snowflake():
    """
    Fetch stock data from Snowflake database.
    """
    conn = return_snowflake_conn()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT SYMBOL, DATE, OPEN, CLOSE, LOW, HIGH, VOLUME
        FROM STOCK.PUBLIC.STOCK_DATA
        WHERE SYMBOL IN ('GOOGL', 'AAPL')
    """)
    rows = cursor.fetchall()
    column_names = [desc[0] for desc in cursor.description]

    # Convert to DataFrame
    df = pd.DataFrame(rows, columns=column_names)

    logging.info(f"✅ Fetched {len(df)} rows from Snowflake.")
    cursor.close()
    conn.close()

    return df

@task(dag=dag)
def forecast_stock_prices(df):
    """
    Forecast stock prices using ARIMA.
    """
    df['DATE'] = pd.to_datetime(df['DATE'])
    df = df.sort_values('DATE')

    forecast_df = pd.DataFrame()

    for symbol in df['SYMBOL'].unique():
        symbol_df = df[df['SYMBOL'] == symbol]  # Filter by symbol
        logging.info(f"Data for {symbol}:\n{symbol_df.head()}")

        # ARIMA model for forecasting
        try:
            model = ARIMA(symbol_df['CLOSE'], order=(5, 1, 0))  # ARIMA model configuration (p, d, q)
            model_fit = model.fit()

            forecast = model_fit.forecast(steps=30)

            last_date = symbol_df['DATE'].max()
            forecast_dates = pd.date_range(start=last_date, periods=30 + 1, freq='D')[1:]

            symbol_forecast_df = pd.DataFrame({
```

```
                'DATE': forecast_dates,
                'SYMBOL': symbol,
                'ACTUAL': [None] * 30,
                'FORECAST': forecast
            })

            forecast_df = pd.concat([forecast_df, symbol_forecast_df])

        except Exception as e:
            logging.error(f"❌ Error forecasting {symbol}: {e}")

    logging.info(f"Forecasted data:\n{forecast_df[['DATE', 'SYMBOL', 'ACTUAL', 'FORECAST']].head()}")

    return forecast_df


@task(dag=dag)
def load_forecast_to_snowflake(df):
    """
    Load forecasted results into Snowflake.
    """
    conn = return_snowflake_conn()
    cursor = conn.cursor()

    # Create or replace the forecast table in Snowflake
    cursor.execute("""
        CREATE OR REPLACE TABLE STOCK.PUBLIC.STOCK_FORECAST (
            SYMBOL STRING,
            DATE DATE,
            ACTUAL FLOAT,
            FORECAST FLOAT
        )
    """)

    for _, row in df.iterrows():
        date_str = row['DATE'].strftime('%Y-%m-%d')  # Convert date to string format
        cursor.execute("""
            INSERT INTO STOCK.PUBLIC.STOCK_FORECAST (SYMBOL, DATE, ACTUAL, FORECAST)
            VALUES (%s, %s, %s, %s)
        """, (row['SYMBOL'], date_str, row['ACTUAL'], row['FORECAST']))  # Pass the formatted date

    cursor.close()
    conn.close()

    logging.info("✅ Forecast data successfully loaded into Snowflake!")


@task(dag=dag)
def run_pipeline():
    """
    Run the entire pipeline (fetch data, forecast, load forecast).
    """
    try:
        stock_data = fetch_stock_data_from_snowflake()
        forecasted_data = forecast_stock_prices(stock_data)
        load_forecast_to_snowflake(forecasted_data)
        logging.info("✅ ETL pipeline completed successfully!")
    except Exception as e:
        logging.error(f"❌ Error in ETL pipeline: {str(e)}")


# Trigger another DAG (stock_price_pipeline) at the start
trigger_stock_price_pipeline = TriggerDagRunOperator(
    task_id='trigger_stock_price_pipeline',
    trigger_dag_id='stock_price_pipeline',  # Name of the DAG to be triggered
    dag=dag,
)

# Define the task sequence in the DAG
start_task = DummyOperator(task_id="start", dag=dag)

# Define tasks
etl_task = fetch_stock_data_from_snowflake()
forecasting_task = forecast_stock_prices(etl_task)
load_task = load_forecast_to_snowflake(forecasting_task)

# Set the sequence of task execution
```

start_task >> trigger_stock_price_pipeline >> etl_task >> forecasting_task >> load_task

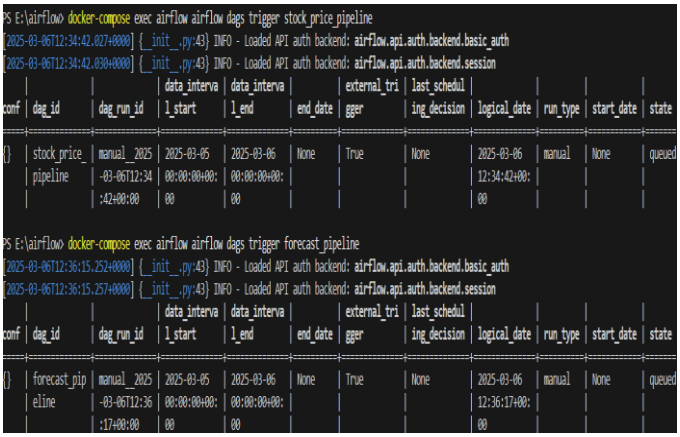
4.3 DOCKER CODES:

To Start: docker compose up

To Stop: docker compose down

To execute Dag's: docker-compose exec airflow airflow dags trigger forecast_pipeline

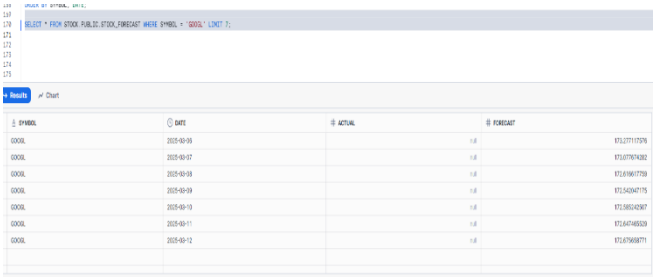
docker-compose exec airflow airflow dags trigger stock_price_pipeline



4.4 SnowFlake Codes

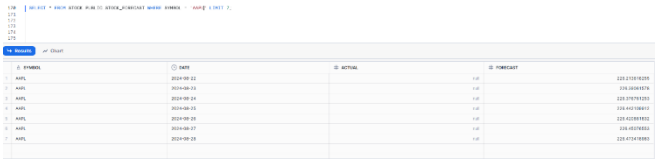
Google Forecast:

SELECT * FROM STOCK.PUBLIC.STOCK_FORECAST
WHERE SYMBOL = 'GOOGL' LIMIT 7;



Apple Forecast:

SELECT * FROM STOCK.PUBLIC.STOCK_FORECAST
WHERE SYMBOL = 'AAPL' LIMIT 7;



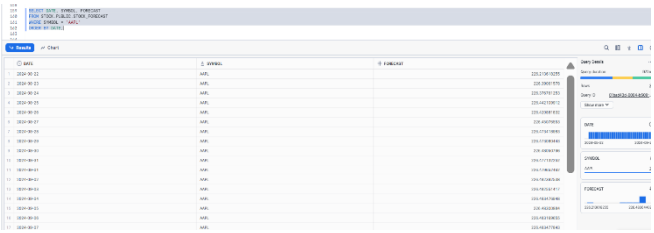
Apple Stocks 180 days data:

SELECT DATE, SYMBOL, FORECAST

FROM STOCK.PUBLIC.STOCK_FORECAST

WHERE SYMBOL = 'AAPL'

ORDER BY DATE;

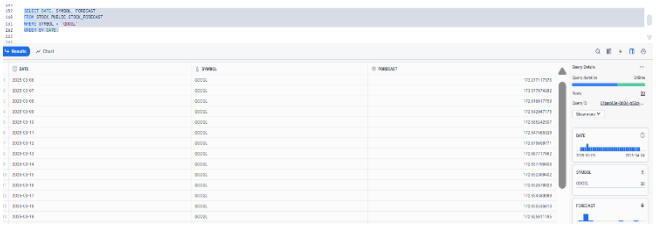


Google Stock's 180 days Data:

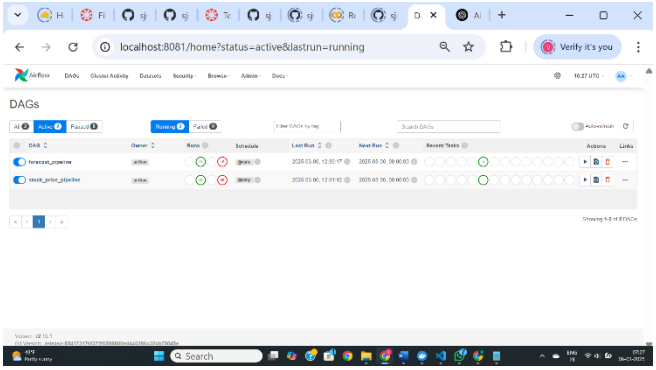
SELECT DATE, SYMBOL, FORECAST

FROM STOCK.PUBLIC.STOCK_FORECAST

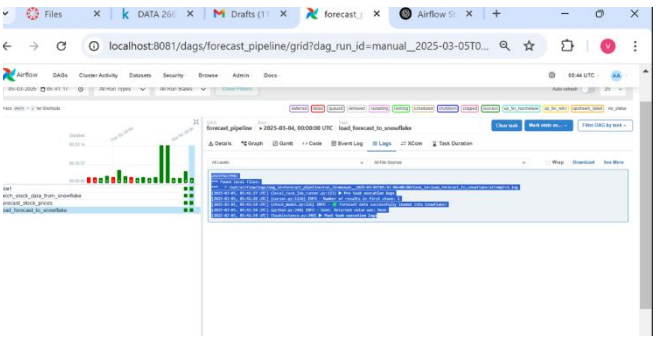
WHERE SYMBOL = 'GOOGL'



4.4 Apache Airflow UI



4.5 Forecast DAG Graph



4.6 Tables Structure:

Stock_Data Table:

```
    CREATE OR REPLACE TABLE STOCK.PUBLIC.STOCK_DATA (
        SYMBOL STRING,      -- Stock symbol (e.g., GOOGL, AAPL)
        DATE DATE,          -- Trading date
        OPEN FLOAT,         -- Opening price
        CLOSE FLOAT,        -- Closing price
        LOW FLOAT,          -- Lowest price of the day
        HIGH FLOAT,         -- Highest price of the day
        VOLUME INTEGER      -- Number of shares traded
);
```

STOCK_FORECAST Table:

```
    CREATE OR REPLACE TABLE STOCK.PUBLIC.STOCK_FORECAST (
        SYMBOL STRING,      -- Stock symbol (e.g., GOOGL, AAPL)
        DATE DATE,          -- Forecasted date
        ACTUAL FLOAT,       -- Actual closing price (if available, NULL otherwise)
        FORECAST FLOAT      -- Forecasted closing price
);
```

STOCK_DATA_STAGING:

```
    CREATE OR REPLACE TABLE STOCK.PUBLIC.STOCK_DATA_STAGING (
        SYMBOL STRING,      -- Stock symbol (e.g., GOOGL, AAPL)
        DATE DATE,          -- Trading date
        OPEN FLOAT,         -- Opening price
        CLOSE FLOAT,        -- Closing price
        LOW FLOAT,          -- Lowest price of the day
        HIGH FLOAT,         -- Highest price of the day
        VOLUME INTEGER      -- Number of shares traded
);
```